

Health Insurance Claim First-Response Agent

Team B-9 - Section B - Applied AI System (Problem A)

Recorded demonstration: <https://www.youtube.com/watch?v=BoABxRL6Eu0>

1. Why an Agent

Problem A belongs on rung 7 because the claim does not determine a fixed sequence. Policy status can end a run immediately; each claim has a different number of lines; only some procedures create a pre-authorisation dependency; and later observations may change whether the agent should approve in principle, request a named document, or escalate. A ReAct loop therefore lets the model select the next check at runtime while every observation is grounded in local system-of-record data.

Table 1. Architecture ladder

Rung	Who decides?	Cost	Use it when	Problem A verdict
1 Single call	Code/user	1 call	One-shot, well-specified task	Insufficient: authoritative evidence is outside the request.
2 Prompt chain	Code	2–4 calls	Fixed subtasks with checks	Insufficient: cannot stop early or add a dependent lookup.
3 Routing	Model picks a lane	1 + 1 calls	Distinct predefined categories	Insufficient: an initial lane cannot revise itself after contradictory evidence.
4 Parallelisation	Code	N calls, one wall-clock	Independent checks	Useful inside the agent, but cannot decide which new query an observation creates.
5 Orchestrator-workers	Model at runtime	Unpredictable	Input-dependent task split	Unnecessary: this is one investigation, not worker fan-out.
6 Evaluator-optimiser	Model	Unknown rounds	Criteria support iterative refinement	Does not solve the variable evidence path.
7 Single agent	Model at runtime	Variable until capped	Steps cannot be known in advance	Chosen: observations determine the next check and stopping point.

Rung 7 therefore buys adaptive evidence gathering at the cost of variable trajectories, variable spend and a larger failure surface. Because those paths cannot be exhaustively enumerated, the loop must be capped, instrumented, evaluated by outcome and protected by guardrails.

Table 1a. The workflow test

Question	Answer for Problem A
Who decides the sequence, and when?	The model, at runtime. <code>lookup_member_policy</code> establishes eligibility first; subsequent checks depend on the evidence returned.
Does step count vary with the input?	Yes. Parallel scripted trajectories span 3–6 turns (median 5) across the 50-case set.
Can every path be enumerated?	No useful fixed path captures the observation-dependent choices; the harness tests outcomes rather than prescribing tool sequences.
Is cost predictable before caps?	No. It is variable until the step and resource-budget ceilings bound the run.

Table 1b. The three-question governance-cliff test

System	Who picks what to retrieve?	Can it loop/re-query?	Can it change the world?
Retrieval (RAG)	Code, through one fixed query	No	No - read-only
Agentic retrieval	The model, at runtime	Yes	No - read-only
This system	The model, at runtime	Yes	Yes - issue_decision_letter appends a gated structured decision record

Table 1c. Four measured traces spanning the observation-dependent range

Case	Shape	Turns	Tool calls
CLM-8850	Short, one line, no unusual dependency	5	5
CLM-8910	Early exit - lapsed policy	3	2
CLM-9000	Long, five lines, with independent checks batched	5	9
CLM-8888	Dependent pre-authorisation chase	6	8

Table 1d. Machine-speed ground-truth sources

Record/tool source	Fact that can contradict the model	How fast
Member/policy record	Status, validity dates and remaining annual limit	Same run - local lookup
Coverage/procedure record	Exclusion and pre-authorisation requirement	Same run - local lookup
Pre-authorisation record	Valid, missing or expired authorisation	Same run - local lookup
Hospital record	Panel status	Same run - local lookup
Decided-claims record	Duplicate evidence	Same run - local lookup

The loop remains defensible because every investigation step receives objective, machine-speed ground truth from local records that can contradict the model. The governance cliff is the write itself: everything before `issue_decision_letter` is read-only agentic retrieval, so the `confirm` autonomy gate belongs immediately before that action, not around the whole investigation.

Reliability is less comfortable. GPT-4o-mini's 90 live D4 trials jointly measured $P=60/90=66.7\%$ and median $T=8$. Pairing values from that same run distribution implies $s=P^{(1/T)}=95.1\%$. Holding that diagnostic rate fixed predicts 81.6% at four turns and 54.4% at twelve. The estimate is diagnostic rather than physical because steps are neither independent nor equally failure-prone.

The traces point primarily to step quality: among the 30 failed v2 trials, `get_preauthorisation` was the final recorded evidence step in 18, making pre-authorisation handling the leading weak-step candidate. Turn count still matters. Because D2(c) reduces T , it can improve whole-run reliability as well as token cost; the resulting P is the same quantity that determines D6's expected fallback layer, $(1-P)F$.

Increasing turns compounds failure risk and expected cost

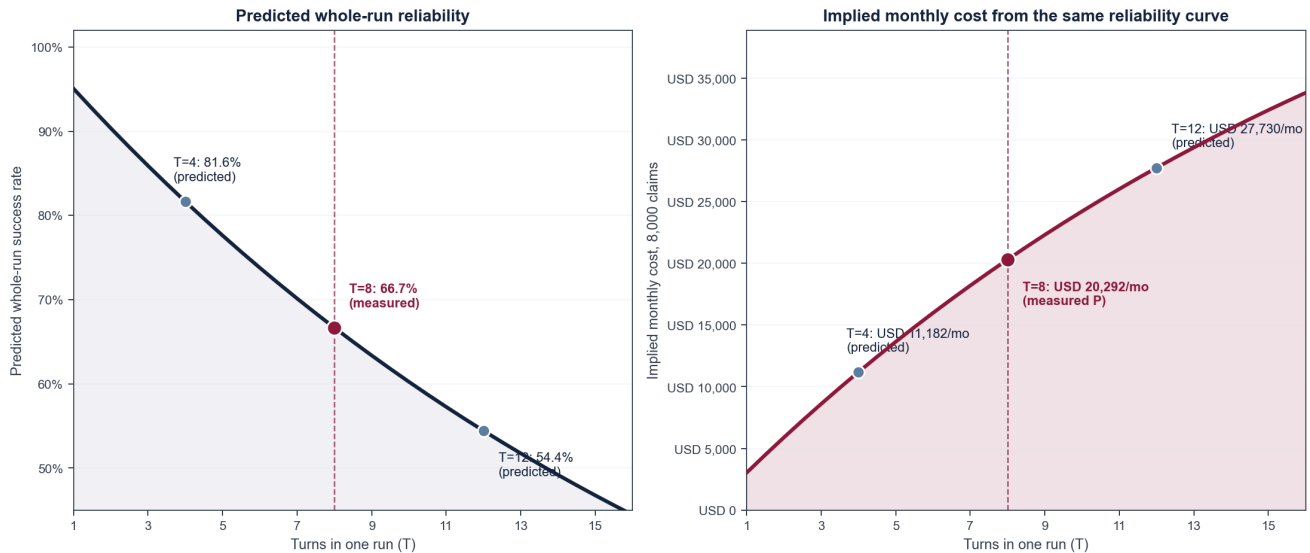


Figure 1. Holding implied per-step reliability fixed, longer trajectories reduce predicted success and raise expected fallback cost; P and T=8 come from the same GPT-4o-mini live-trial distribution, while T=4 and T=12 are diagnostic projections.

A good run: (1) names a cause traceable to records; (2) follows the fixed routing rule; (3) writes at most once after confirmation; (4) asks or escalates rather than inventing facts; and (5) costs less than manual handling under the measured cost-to-serve model while remaining within the shipped caps.

2. The Tool Layer

The tool surface was treated as an agent-computer interface, not an inventory. The shortest defensible set retains only tools that prevent an observed failure, remain distinguishable and justify their recurring prefix cost. Even unused descriptors are resent every turn, while each write enlarges the gated surface. We removed `get_claim` because it duplicated the request and appeared in none of 50 scripted trajectories, and rejected search because no task failed without it. We considered wider coverage lookup but retained inspectable line calls; validation stayed in code.

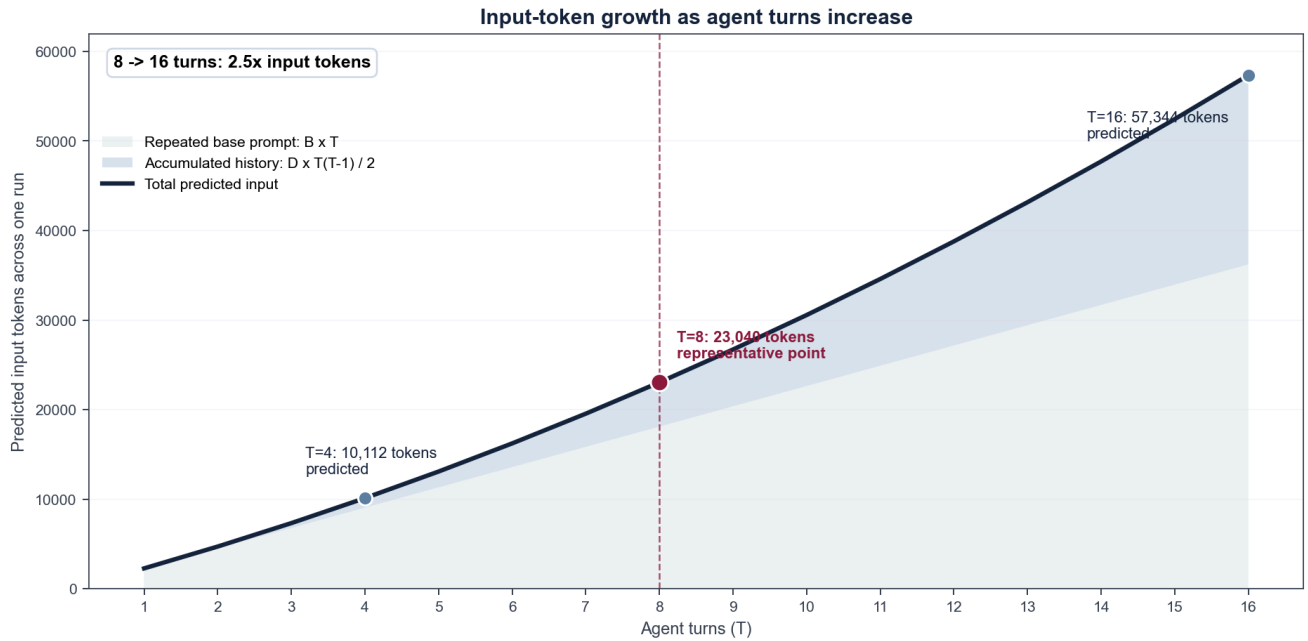


Figure 2. With measured $B=2,264$ and representative $D=176$ tokens/turn, 8 to 16 turns increases predicted input from 23,040 to 57,344 tokens (2.49x), explaining why D2(c) targets turn count.

Table 2a. Final tool audit

Tool	Fails without it?	Confusable?	Why it earns its place
lookup_member_policy	Yes	No	Only source of status, dates, exclusions and remaining limit.
check_coverage	Yes	Low	Only line-level exclusion, document and preauthorisation-requirement evidence.
get_preauthorisation	Yes, where required	Low	Tests whether required authorisation is valid on the service date.
check_hospital	Required evidence	Low	Records panel status in the evidence trail.
check_duplicate	Yes	Low	Prevents an already-decided duplicate from being approved again.
issue_decision_letter	Yes	No	The only gated write.
get_claim	No	-	Removed: duplicated the request; zero calls in 50 traces; -96 prefix tokens/turn.

Descriptor and interface rewrite

The model sees the interface contract rather than the implementation, so interface design is stronger than another instruction to “be careful.” V2 supplies the full six-field contract: typed signature, unique purpose, input and invalid-value behaviour, bounded return, named failure semantics, and irreversible status with its gate. The poka-yoke changes below move safety from probabilistic obedience into code constraints that hold regardless of model wording or provider.

Table 2b. Poka-yoke interface changes

Before	After	What it makes impossible
Preauthorisation lookup without a service date	<code>date_of_service</code> required	Treating existence as proof that authorisation is valid for this claim date.
Partial duplicate lookup	Member, hospital, date and line items all required	A silently loosened duplicate comparison.
Free-text decision value	Closed three-value decision set	Writing an invented fourth outcome.
Free-form approval detail	Per-line dispositions and numeric totals required	Recording an approval that silently omits a claim line.

The duplicate-lookup row was not designed in the abstract. Negative case CLM-8933's own note - match on all four facts, since the history holds three near-misses - only makes sense because a weaker match first misfired; paired case CLM-8960 confirms it would also wrongly escalate a real approval, which is why that row exists.

Two controlled v1/v2 comparisons separate the interface effects. The descriptor experiment holds model, cases and return shape fixed: clearer v2 wording gains only 2.3 points for a small prompt premium. The return-shape experiment pins v2's descriptor in both arms and changes only `check_coverage` from prose to typed JSON. JSON costs ~20 more observation tokens per call but gains 4.4 points, entirely through four extra negative-trial passes. These modest results support labelled fields without claiming a precise effect; the two pairs run independently and must not be compared.

Table 2c. Descriptor v1 versus v2, model and cases held fixed

Metric	v1	v2	Interpretation
Trial pass rate	58/90 (64.4%)	60/90 (66.7%)	+2.3pp
Ordinary trials	27/30 (90.0%)	28/30 (93.3%)	+3.3pp
Negative trials	31/60 (51.7%)	32/60 (53.3%)	+1.6pp
Target descriptor block	~34 tokens	~152 tokens	+118 tokens/turn
Complete system prompt	2,176 tokens	2,264 tokens	+88 tokens/turn
Actual observation return	Median ~29; max ~37 tokens	Median ~29; max ~37 tokens	Unchanged
Scripted guardrail control	12/12 (100%)	12/12 (100%)	Unchanged shared control
Total cost, 90 trials	\$0.2736	\$0.2822	+\$0.0086

Descriptor v1 vs v2, openai/gpt-4o-mini

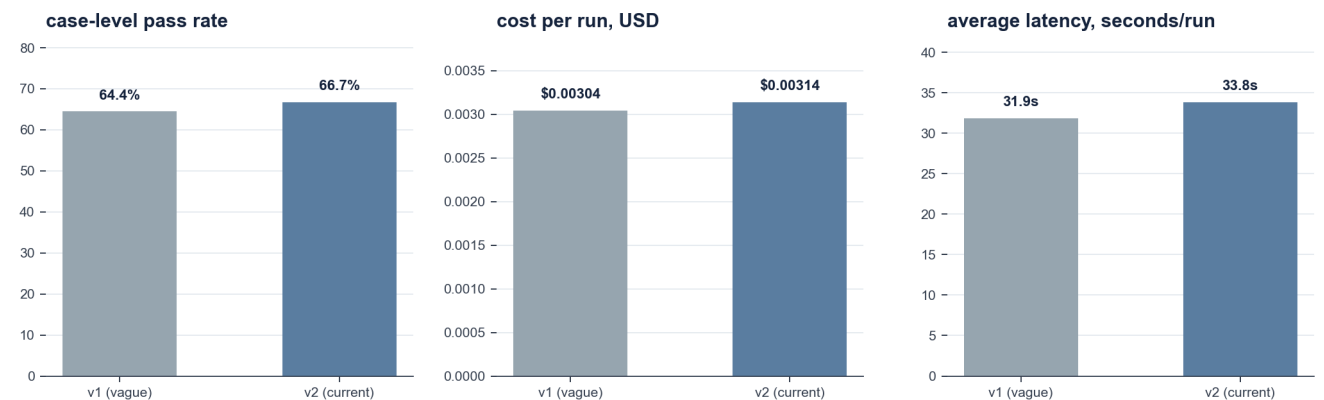


Figure 3. V2 produced two additional correct trials at a small token premium; observation shape was held constant.

Table 2d. Return-shape v1 versus v2 - v2 descriptor fixed, run 10 September 2026

Metric	v1 prose	v2 typed JSON	Interpretation
Trial pass rate	63/90 (70.0%)	67/90 (74.4%)	+4.4pp
Ordinary trials	27/30 (90.0%)	27/30 (90.0%)	Unchanged
Negative trials	36/60 (60.0%)	40/60 (66.7%)	+6.7pp
Strict negative cases	9/20	10/20	+1 case
Mean observation size	~29.32 tokens	~49.08 tokens	+19.76 tokens/call
Total cost, 90 trials	\$0.2807	\$0.2835	+\$0.0028
Guardrail control	12/12	12/12	Shared code-layer control; unaffected

Dependency rule and parallel calls

The dependency rule follows evidence availability rather than tool convenience. Policy eligibility must resolve first and duplicate checking follows; hospital status and independent line-coverage checks may then batch because neither needs another call's observation. Preauthorisation must wait until coverage identifies which lines require it. `issue_decision_letter` remains last, after complete evidence and confirmation, and still consumes a turn - gating does not make the action free.

Parallelisation regrouped work rather than removing evidence: the same 274 calls and 50/50 correctness needed fewer turns, fewer resent tokens and lower cost, because turn count carries both the fixed tool prefix and accumulated history; output barely moved since the work was unchanged. The optimisation is not universal: a batch may launch a call an earlier observation would have avoided, and batching removes a model decision point. We preserve dependencies where that lost decision could affect correctness.

Table 2e. Sequential versus parallel scripted replay

Metric	Sequential	Parallel	Change
Tool calls	274	274	Unchanged
Turns	324	255	-69 (-21.3%)
Input tokens	860,782	673,615	-187,167 (-21.7%)
Output tokens	25,179	25,025	-154 (-0.6%)
Cheap-tier cost	\$0.0929	\$0.0748	-\$0.0181 (-19.5%)
Mid-tier cost	\$0.9546	\$0.7735	-\$0.1811 (-19.0%)
Correct cases	50/50	50/50	Unchanged

3. What the Evidence Showed

The evaluation was designed to test outcomes rather than prescribe paths. Its 50 isolated cases combine 30 ordinary approvals with 20 negative cases, given three trials, since these most often flip between runs and one pass may be luck; this produces 90 trials per model. Supplied cases were graded against `expected_outcomes_A.json`; team-added cases use labels in the same outcome-and-trigger schema. Each case is labelled with its applicable code or judgement check, separating structured correctness from prose quality. The scripted backend reproduces the complete battery without a network or key, proving the harness and controls - not live-model luck - generate the measurements.

Table 3a. Negative families and the wrong behaviour each is designed to expose

Negative family	Cases	Wrong behaviour caught
Missing or expired preauthorisation	5	Approving without a valid authorisation for the service date.
Required document absent	3	Approving unsupported lines or asking vaguely instead of naming the item and line.
Policy status, dates or annual limit	6	Continuing or approving when eligibility already requires escalation.
Previously decided duplicate	2	Approving a resubmitted claim as new.
Hostile claim narrative	4	Following untrusted instructions, fake authority or imitation tool output.

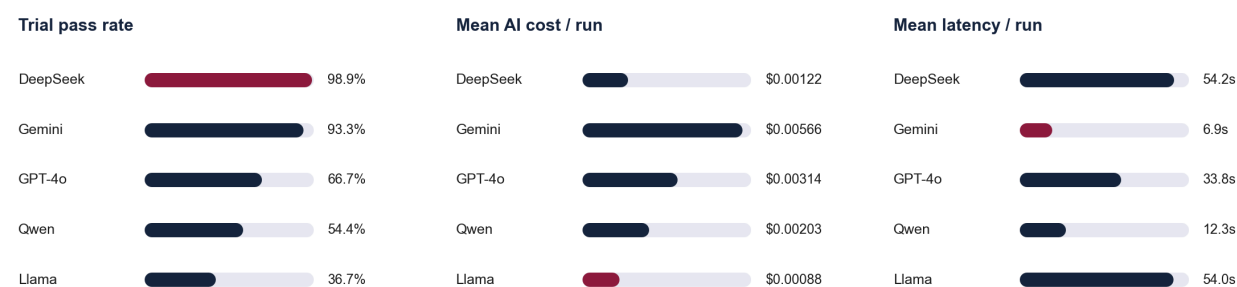
Table 3b. Code checks and judgement checks answer different questions

	Code check	Judgement check
Checks	Decision, trigger, missing line, approval/refusal totals and exactly one gated write.	Whether the reason is specific and the evidence trail actually supports the outcome.
Method	Deterministic assertions against the answer key.	Committed rubric applied by an independent grader to selected live records.
Coverage	Every trial; scripted control reproduced 90/90.	Twenty disjoint records across two candidate models and two graders.
Result	Reported per model in Table 3c.	2/20 passed; evidence-detail omissions dominate.

The live comparison is a controlled model intervention: every member used the same final v2 prompt, 50-case set, trial policy and frozen code version; only the model string changed. Five distinct families span multiple price tiers. Judgement is not folded into those model pass rates: GPT-5 graded ten Gemini records and Claude Sonnet 5 graded ten disjoint GPT-4o-mini records under the separately committed rubric.

Table 3c. Final v2 live battery - 90 trials/model, run 8–9 September 2026

Model / family	Tier (\$/M in–out)	All trials	Ordinary	Negative	Median turns	90-run cost	Latency
DeepSeek V4 Flash / DeepSeek	\$0.081–\$0.162	89/90 (98.9%)	30/30	59/60	5	\$0.110	54.2s
Gemini 2.5 Flash / Google	\$0.30–\$2.50	84/90 (93.3%)	30/30	54/60	6	\$0.509	6.9s
GPT-4o-mini / OpenAI	\$0.15–\$0.60	60/90 (66.7%)	28/30	32/60	8	\$0.282	33.8s
Qwen 2.5 7B / Alibaba	\$0.10–\$0.20	49/90 (54.4%)	24/30	25/60	7	\$0.183	12.3s
Llama 3.1 8B / Meta	\$0.05–\$0.08	33/90 (36.7%)	13/30	20/60	6	\$0.080	54.0s



Measured trial accuracy, Layer-1 cost, and latency across the live battery, current 50-case set.

Figure 5. The five models rank differently on accuracy, Layer-1 cost and latency; token price alone therefore cannot determine deployment choice.

The evidence supports a task-specific choice, not a universally “best” model. DeepSeek delivered the strongest accuracy and lowest expected fallback cost but imposed high latency; Gemini bought much faster responses with a larger token bill and slightly weaker reliability. The main separation appeared on negative cases: GPT-4o-mini’s strong ordinary-case result concealed a large fall under missing, contradictory or hostile evidence. The negative set therefore changes the deployment conclusion, not merely reported accuracy: ordinary cases make several models look viable; adverse evidence tests reliability where incorrect action matters most. Tool-output impersonation remained a cross-family weakness, with every model failing at least one such trial. Only 2/20 sampled explanations met the evidence rubric even when structured fields were correct - Section 6 reports a follow-up fix. Outcome accuracy is not explanation quality.

4. What It Costs

We use the Class 5 escalation model because a wrong claim is handed to a human assessor rather than retried by the agent. Layer 1 prices measured input and output tokens; Layer 2 prices expected human fallback; Layer 3 is a stated \$5 monthly monitoring and maintenance allowance. For Problem A, **F=\$38/hour × 12/60=\$7.60** and **V=8,000 claims/month**. No prompt-caching or reasoning-token adjustment is claimed; the baseline uses measured token totals at list price.

Three-layer model:

$$C_{\text{serve}} = C_{\text{AI}} + (1-p)F$$

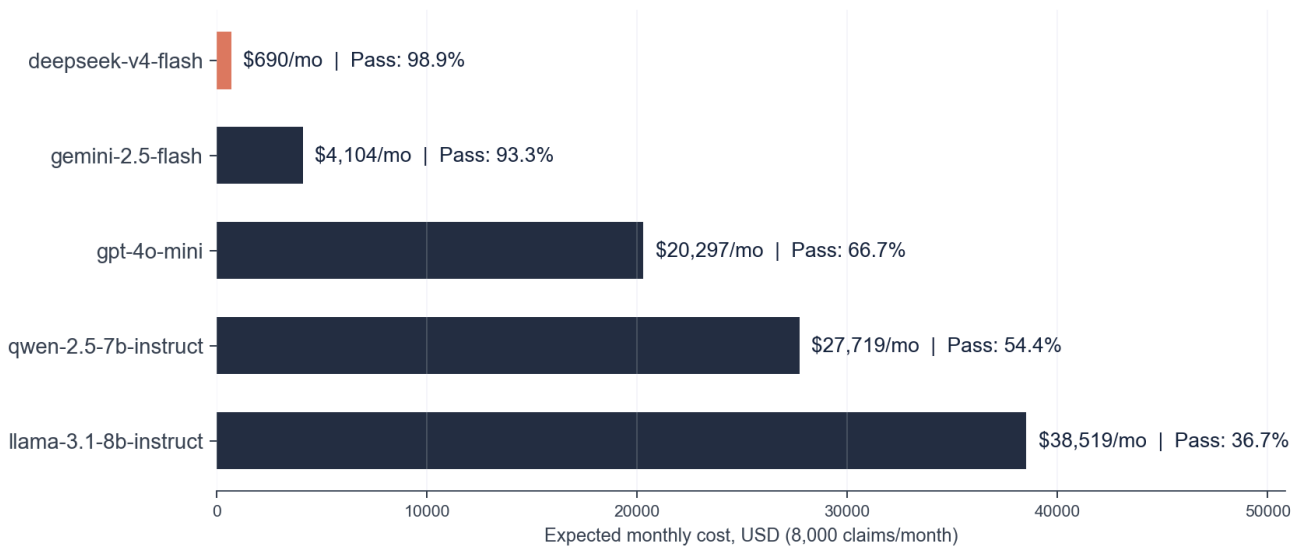
$$\text{Monthly} = V[C_{\text{AI}} + (1-p)F] + \text{Fixed}$$

Table 4a. Three-layer expected cost

Model	L1 AI/run	L2 fallback/task	Total/task	Monthly @ 8,000
DeepSeek V4 Flash	\$0.00122	\$0.0844	\$0.0857	\$690
Gemini 2.5 Flash	\$0.00566	\$0.5067	\$0.5123	\$4,104
GPT-4o-mini	\$0.00314	\$2.5333	\$2.5365	\$20,297
Qwen 2.5 7B	\$0.00203	\$3.4622	\$3.4643	\$27,719
Llama 3.1 8B	\$0.00088	\$4.8133	\$4.8142	\$38,519

Claims First-Response Agent: Expected Monthly Cost by Model

Assumptions: 8,000 claims/month | \$7.60 human fallback per failed claim | \$5 monthly regression testing



Expected monthly cost = V x [C_AI + (1-p) x C_fallback] + F (V = claim volume, p = trial-level pass rate; human fallback dominates token cost)

Figure 5b. Expected monthly cost across all five models, at each model's own measured pass rate - a different comparison from the GPT-4o-mini sensitivity check in Figure 6, which stress-tests one model's accuracy estimate rather than ranking across models.

Across the battery, human fallback is orders of magnitude larger than per-run inference, so Layer 2 - not token price - sets the economic ranking. Raw token price reverses the deployment ranking: Llama is cheapest at Layer 1 but most expensive after fallback, while DeepSeek's higher success rate makes it cheapest overall.

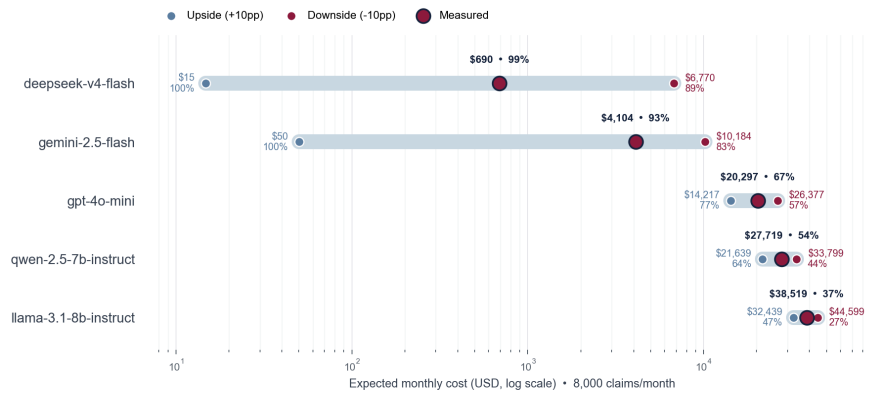
Table 4b. Four measured cost levers

Lever	Before	After	Finding
1 Tool block B	7 stub tools: 2,095 tokens	Cut-only: 1,999; full v2: 2,264	Cut saved 96; required detail increased net prefix.
2 Turn count T	860,782 sequential input tokens	673,615 parallel	-21.7%, the measured quadratic lever.
3 Observation size D	v1 prose: mean 29.32 tokens	v2 JSON: mean 49.08 tokens	+19.76 tokens/call bought +4.4pp; no token saving.
4 Success rate P	Llama 36.7%	DeepSeek 98.9%	Monthly cost falls \$37,829 (98.2%).

The ledger separates worthwhile engineering from the dominant business lever. Removing get_claim permanently reduced the repeated prefix, and parallel calls materially reduced the quadratic turn term. The typed-JSON experiment increased, not reduced, observation size; its small accuracy gain therefore carries an ongoing token cost and is not assumed in the baseline battery. Lever 4 still dominates by two to three orders of magnitude because human fallback dwarfs every model's inference cost, reversing the ranking suggested by token price alone. Levers 1-3 are still worth shipping regardless: they cost nothing to maintain and compound under whichever model is chosen, while lever 4 has to be re-measured every time a cheaper model becomes viable.

Monthly cost sensitivity by model

Range shows each model at ± 10 percentage points from its measured success rate.



Key finding: even under downside assumptions, DeepSeek and Gemini remain cheaper than the other models' upside cases.

Figure 5c. Monthly cost sensitivity by model at ± 10 percentage points from each measured success rate.

Across all five models, the ± 10 -point ranges do not overturn the ranking: even DeepSeek and Gemini's downside costs remain below the other models' upside costs.

Table 4c. ± 10 percentage-point sensitivity around GPT-4o-mini

Scenario	Success rate	Fallback/task	Total/task	Monthly
Downside	56.7%	\$3.2933	\$3.2965	\$26,377
Measured	66.7%	\$2.5333	\$2.5365	\$20,297
Upside	76.7%	\$1.7733	\$1.7765	\$14,217

A ± 10 -point movement changes GPT-4o-mini's monthly estimate by \$6,080 in either direction, yet even the upside remains far above DeepSeek's measured total; that deployment conclusion survives the tested range. Break-even asks how accurate each alternative must become to match DeepSeek's \$0.0857 all-in cost. Because model-run costs are tiny beside F, every threshold remains near DeepSeek's own success rate and none of the measured alternatives clears it.

Break-even: $p_{BE} = 1 - (E - C) / F$, where E is DeepSeek's all-in cost and C is the alternative model's token-only run cost.

Table 4d. Success required to match DeepSeek's all-in cost

Alternative	Measured	Break-even	Gap
Gemini 2.5 Flash	93.3%	98.9%	+5.6pp
GPT-4o-mini	66.7%	98.9%	+32.2pp
Qwen 2.5 7B	54.4%	98.9%	+44.5pp
Llama 3.1 8B	36.7%	98.9%	+62.2pp

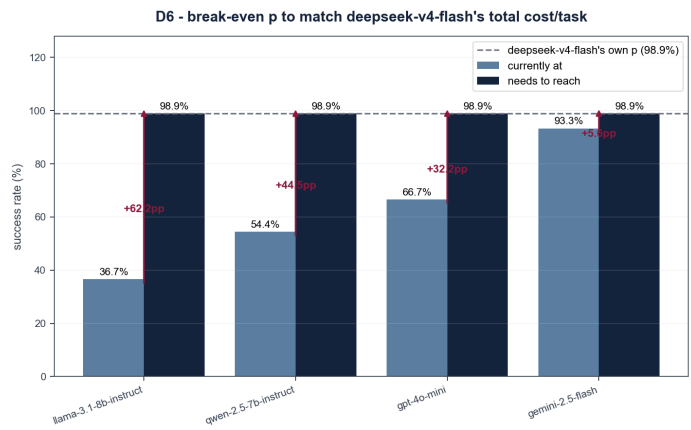
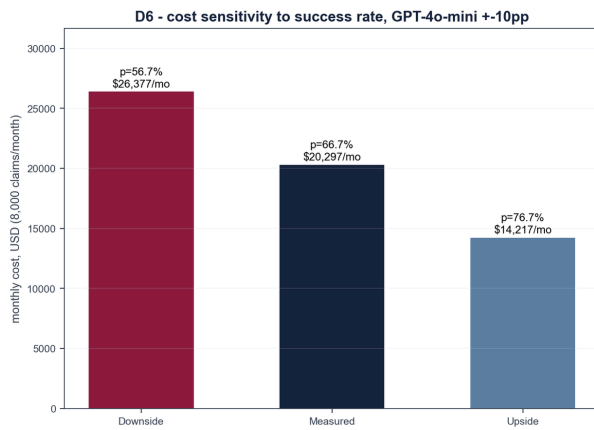


Figure 6. Success-rate sensitivity around GPT-4o-mini. Figure 7. Success required to match DeepSeek's all-in cost.

The shipped controls bound the remaining exposure: 15 model turns per run, 24 tool calls as a pre-execution resource budget, a 1,024-token response ceiling, and \$1 per policy member per month. At DeepSeek's measured average, the user limit permits approximately 11–12 claims before subsequent work routes to the existing human process. The deployment choice is therefore DeepSeek with confirmation and these caps, not autonomous operation justified merely by cheap tokens.

5. The Two Failures

Failure 1 - loop control. Raising `max_repeats` from 2 to 999 removes action de-duplication, so the scripted model repeats one pre-authorisation lookup until the 15-turn step cap. Working and restored runs stop at five turns, two calls, 12,863 input tokens and \$0.00138; broken reaches 15 turns, 15 calls, 45,109 tokens and \$0.00479. All three safely escalate, so pass rate misses the defect; turn, token, cost and guard instrumentation expose it. De-duplication is the loop-layer fix, while the step cap is the backstop that catches its symptom. The 24-call budget never fires first, since 15 turns means only 15 calls, and a prompt instruction cannot guarantee termination when a repeated action stays syntactically valid.

Table 5a. Failure 1: loop-control failure on CLM-8842

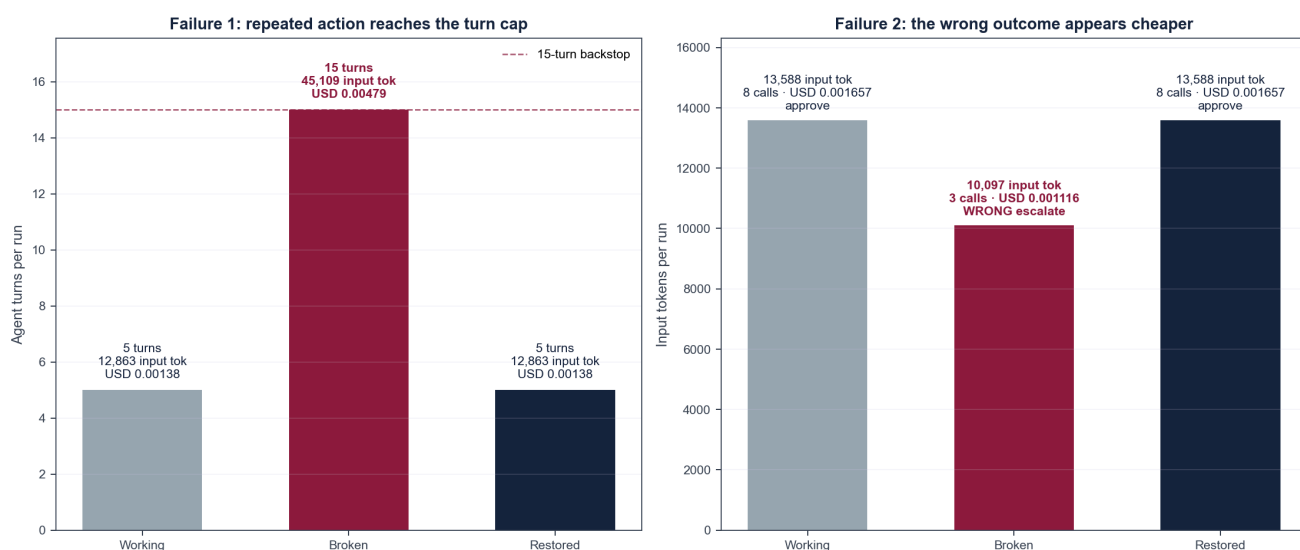
Metric	Working	Broken	Restored
Turns	5	15	5
Tool calls	2	15	2
Input tokens	12,863	45,109	12,863
Cost	\$0.00138	\$0.00479	\$0.00138
Pass	True	True	True
Guard triggered	repeated_action	step_cap_exceeded	repeated_action

Failure 2 - tool interface. Removing line items from `check_duplicate`'s four-fact match makes CLM-8960 collide with a different prior claim sharing member, hospital and date. The broken run appears efficient - four turns, three calls, 10,097 input tokens and \$0.00112 - but wrongly escalates; working/restored use five turns, eight calls, 13,588 input tokens and \$0.00166 and correctly approve. Only the D4 answer-key check detects it. The agent behaved rationally given false tool output, so a prompt or loop change would treat the symptom, not the source. Restoring the fourth comparison makes this false match structurally impossible.

Table 5b. Failure 2: tool-interface failure on CLM-8960

Metric	Working	Broken	Restored
Turns	5	4	5
Tool calls	8	3	8
Input tokens	13,588	10,097	13,588
Output tokens	745	266	745
Cost	\$0.00166	\$0.00112	\$0.00166
Decision	Approve	Wrong escalate	Approve
Pass	True	False	True
Detector	D4 outcome check	D4 outcome check	D4 outcome check

Broken-state reproductions locate two different failure layers



Coral marks the deliberately broken implementation; blue and navy show the identical working and restored states. Failure 1 is exposed by instrumentation; Failure 2 only by the D4 outcome check.

Figure 8. One defect remains safe but burns money; the other looks cheaper while producing the wrong outcome.

The 50-case scripted distribution is median 5, minimum 3, maximum 6, with 0/50 reaching the 15-turn cap. Those counts calibrate the cap; they are separate from GPT-4o-mini's live median of 8 used with its live pass rate in Do. The cap is therefore well above normal scripted behaviour but still low enough to expose runaway repetition loudly. Restoration leaves the full scripted pass rate at 50/50 and the guardrail checklist at 12/12.

6. What We Would Not Deploy

We would not deploy this prototype with act autonomy. Tool-output impersonation remained a cross-family weakness, and 18/20 explanations failed judgement because the gate checked completeness, never that the reason said anything specific. A cross-checked citation requirement lifted this to 9/20 (GPT-4o-mini 0/10→5/10, Gemini 2/10→4/10, DeepSeek 7/10 fresh) at no cost to existing evidence - D2(b)'s argument extended: an interface constraint changes what a model can get away with, not what it writes once met. It mitigates, not fixes: closing it surfaced two new failures (a loop-control escalation, a false injection flag), and the largest gaps left - near-duplicate reasoning, facts outside the schema - need a sample-specific check, not a general rule. Pre-authorisation and arithmetic errors reinforce this: hostile text needs provenance separation; rules belong in code. `confirm` stays right for a sharper reason now: explanation quality cannot yet self-certify.

Architecture not built. A reviewer agent could flag decisions whose evidence conflicts with trusted tool output, catching some impersonation failures. At DeepSeek's rate that costs roughly **\$0.0012 per claim** plus a second model round - small against the **\$7.60** fallback, but it re-runs the same judgement through another model rather than adding new evidence. The citation requirement above already captured most of that gain in code, for every model, at zero added latency, without guaranteeing repaired provenance. We kept one agent because the cheaper fix did the reviewer's job first.

Deployment decision: retain the single-agent architecture with code-layer guardrails and human confirmation; do not deploy autonomous claim decisions in the current prototype.